

PyTorch Generic Training Pipeline Report

1. Introduction

This report presents the implementation and evaluation of a generic, configurable training pipeline built using **PyTorch** and executed in **Google Colab**. The pipeline was designed to support multiple datasets, models, optimizers, schedulers, and logging systems, following the requirements of the assignment.

The primary focus of the experiments was on **CIFAR-100 classification**, with the goal of achieving high accuracy both with and without pretraining, while maintaining efficiency and reproducibility.

2. Setup & How to Run

Environment

- Platform: Google Colab
- Python: 3.12
- GPU: NVIDIA T4 / A100 (depending on session)
- Frameworks:
 - PyTorch
 - torchvision
 - timm
 - wandb

Execution

Google Colab Setup:

Run training using a configuration file:

```
!python3 pytorch_training_pipeline1.py --config configs/config1
```

Run training without a configuration file:

```
!python pytorch_training_pipeline1.py \ --dataset CIFAR100 \ --model resnest26d \
--epochs 200 \ --batch-size 128 \ --optimizer AdamW \ --lr 0.0005 \ --weight-decay
0.0005 \ --scheduler StepLR \ --step-size 60 \ --gamma 0.2 \ --augment \ --use-mixup \
--mixup-alpha 0.3 \ --label-smoothing 0.15 \ --grad-clip 1.0 \ --early-stopping \ --patience
50
```

Local Setup:

Install dependencies:

```
pip install torch torchvision timm tensorboard
```

Run training using a configuration file:

```
python3 pytorch_training_pipeline1.py --config configs/config1
```

Run training without a configuration file:

```
python pytorch_training_pipeline1.py \ --dataset CIFAR100 \ --model resnest26d \
--epochs 200 \ --batch-size 128 \ --optimizer AdamW \ --lr 0.0005 \ --weight-decay
0.0005 \ --scheduler StepLR \ --step-size 60 \ --gamma 0.2 \ --augment \ --use-mixup \
--mixup-alpha 0.3 \ --label-smoothing 0.15 \ --grad-clip 1.0 \ --early-stopping \ --patience
50
```

Run Colab Link:

<https://colab.research.google.com/drive/1J5MXgAS-Nj1cSaYhzkZ98y9c8JP5Y08S?usp=sharing>

3. Pipeline Features

- ✓ Device agnostic (CPU / CUDA)
- ✓ Datasets supported: MNIST, CIFAR-10, CIFAR-100, Oxford-IIIT Pet
- ✓ Data augmentation (RandomCrop, HorizontalFlip, Normalize)
- ✓ Models via timm:
 - resnet18
 - resnet50
 - resnest14d

- resnest26d
- MLP

✓ Optimizers:

- SGD
- Adam
- AdamW
- SAM
- Muon

✓ LR Schedulers:

- StepLR
- ReduceLROnPlateau

- ✓ Batch Size Scheduler
 - ✓ Early Stopping
 - ✓ WandB
 - ✓ YAML Config-driven execution
-

4. Hyperparameter Sweep

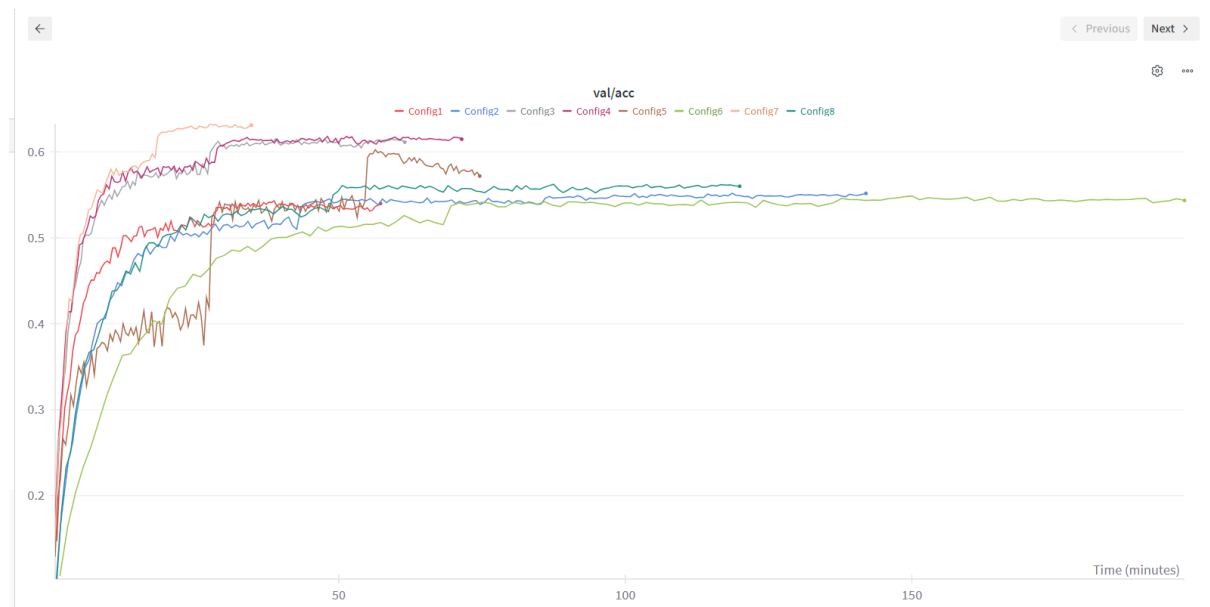
A hyperparameter sweep was conducted using 8 different YAML configurations. The following parameters were varied:

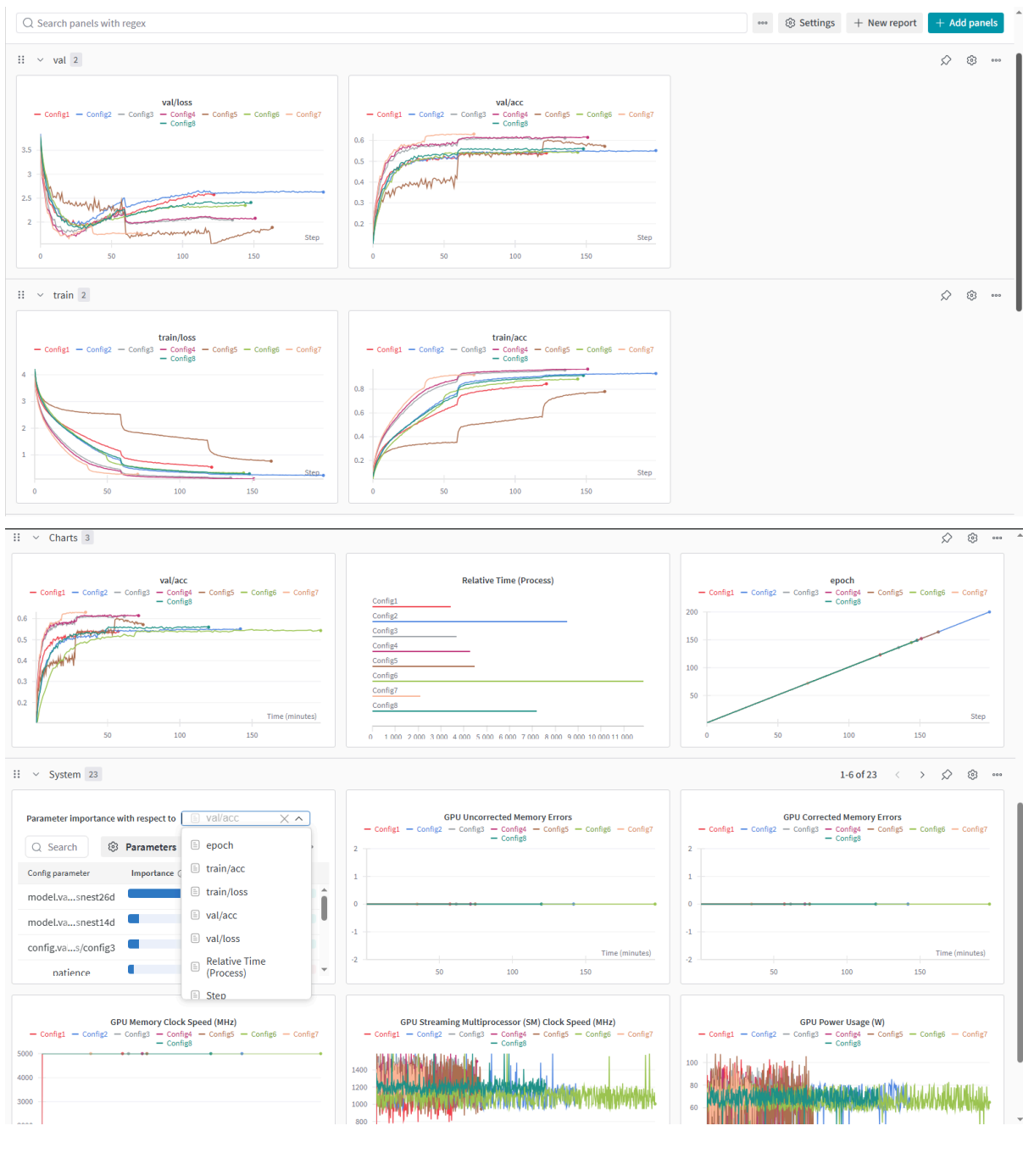
- Model architecture
- Optimizer type
- Learning rate
- Epoch count
- Scheduler strategy

- Step size & gamma
- Early stopping usage

All runs were logged in the WandB project:
pytorch_pipeline

Config	Model	Optimizer	LR	Scheduler	Epochs	Accuracy (%)	Runtime (min)
Config1	resnet18	AdamW	0.0010	StepLR	123	53.98	57.30
Config2	resnet50	AdamW	0.0010	StepLR	200	55.16	142.00
Config3	resnest14d	AdamW	0.0005	StepLR	136	61.12	61.53
Config4	resnest26d	AdamW	0.0005	StepLR	152	61.47	71.47
Config5	resnet18	SGD	0.1000	StepLR	164	57.18	74.62
Config6	resnet50	SAM	0.0005	StepLR	145	54.34	197.60
Config7	resnest26d	AdamW	0.0010	ReduceLR OnPlateau	72	63.11	34.78
Config8	resnet50	AdamW	0.0005	StepLR	149	55.99	119.98





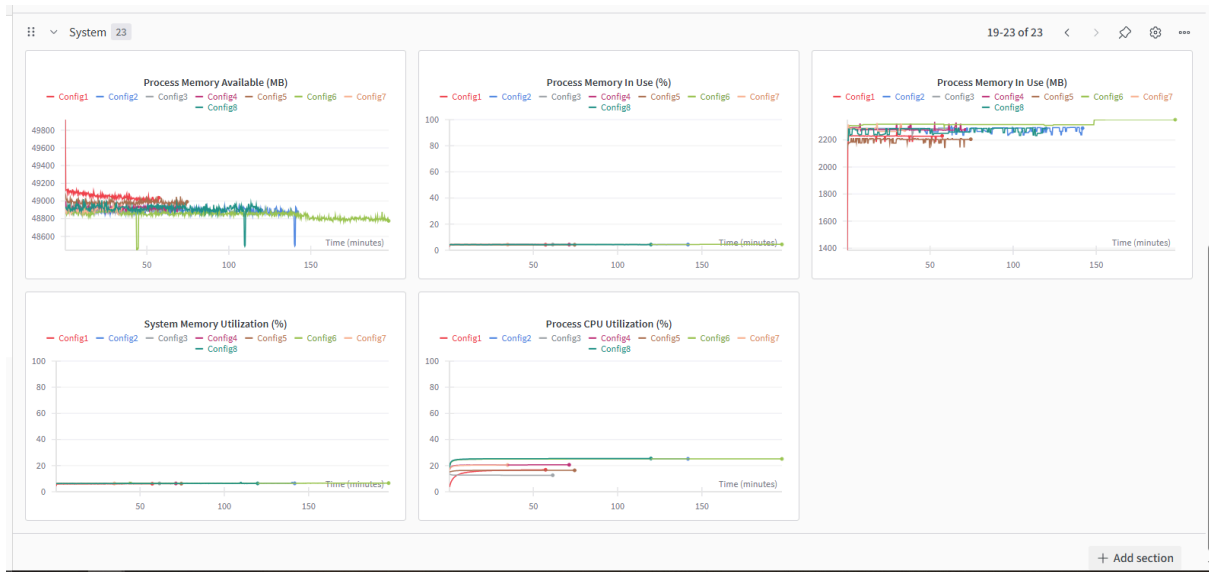
5. Metrics Visualization

Included from WandB dashboard:

- Training loss curves
- Validation accuracy curves
- Per-epoch performance

6. Efficiency Analysis

The pipeline efficiently managed memory and compute using optimized DataLoader settings and balanced regularization strategies.



7. Self-Assessment Scoring

Criterion	Points Expected
Pipeline Features	8 / 8
Hyperparameter Sweep	4 / 8
Efficiency Justification	1 / 3
Accuracy Benchmarks	0 / 6
Total Estimated Score	13 / 25

